Selective tool-knowledge adaptation under environment change

PE6203 Group Project 4 | Integrated final report | 29 October 2026

Abstract. We compare no update, full rewrite and dependency-aware selective patching on 18 fixed MCPEvol-Bench tasks across Early, Middle and Late tool environments, using free local models without fine-tuning. Three rounds produced 486 scored episodes, retaining all 14 execution exceptions. The first added round has actual resource-list differences and is reported separately. Only the original and second added rounds form a 324-observation descriptive comparison, with 18 distinct tasks and two observations per task/policy/version combination. In that comparison, selective patching attains mean ECS 6.242, versus 5.605 for no update and 5.289 for full rewrite. It uses 65.1% fewer update tokens, but 5.6% more recorded total tokens than full rewrite. Recovery and retention remain mixed, and high local scores sometimes coexist with failed calls or absent outputs. The supported finding is lower tool-note regeneration work, rather than a universal quality or total-cost advantage. We disclose input differences, judge limitations and provenance exceptions, and retain negative results.

1. Problem and contribution

An enterprise agent can lose capabilities when a tool is renamed, removed, changes its required arguments or adopts different usage rules. Old harness knowledge may then cause invalid calls even though model weights remain unchanged. Updating everything costs computation and may remove useful guidance for tools that did not change.

We implement selective adaptation of tool contracts and usage notes. Public contract differences identify the initial update set; explicit references to affected tool names expand that set. Only impacted usage notes are regenerated. We compare this method against unchanged knowledge and full regeneration through the same actor loop and real archived MCP servers. The contribution is a working, auditable adaptation pipeline and a bounded empirical comparison, not a new foundation model or a benchmark record.

2. System and three policies

Knowledge contains executable tool contracts and usage notes. A contract includes the public name, description and input schema. Server-derived prefixes disambiguate names. Early notes are the original tool descriptions. At an environment transition, added, removed or modified contracts form the initial affected set. Old/current contracts and previous notes are scanned for exact tool-name references; referring tools join the set until closure. Both raw and prefixed names are recognized.

Policy	Contracts after a change	Usage notes after a change
No update	Keep Early contracts	Keep Early notes
Full rewrite	Copy current authoritative contracts	Regenerate all scoped notes
Selective patch	Copy current authoritative contracts	Regenerate affected and explicitly linked notes; preserve the rest

The two updating policies share the model, updater prompt and output validation. The updater receives requested tool names and current contracts, not the task question, reference answer, mutation labels or scores. Exact-name/nonempty-string validation rejects invalid note generations, retains prior knowledge and records consumed tokens. It does not establish semantic correctness. Copying schemas prevents generated notes from redefining parameters; actor planning can still fail.

The dependency graph is lexical. It can miss implicit workflow prerequisites and can expand unnecessarily when a name is only mentioned descriptively. A supplementary component comparison disables this expansion. We do not claim that name/schema checking alone understands operation order or identifier provenance.

The actor permits at most 12 model-decision rounds; one round may contain several tool calls. It receives the task, actual resource listing and policy-specific knowledge, calls current tools and saves raw requests/responses, returns, errors and file changes. A common output-verification opportunity uses an ordinary round. Whole old assistant/tool groups may be pruned under actor context pressure, with pruning recorded. Episode workspaces are isolated; knowledge carries across versions within a task-policy sequence, while task execution state does not. Judge scoring follows execution, preserving a separate actor receipt.

3. Benchmark, models and completed experiments

MCPEvol-Bench is pinned to 888c779a0fe735f612ca62bdd1fdd7c9e6bf8701. Early, Middle and Late use original, iter3 and iter5 server archives. Eleven local MCP packages and dependencies were extracted unchanged; all 32,388 extracted files were compared against the official archives. A Windows import loader supplies file-URL compatibility without modifying tool implementations.

The fixed test IDs are 9, 28, 103, 105, 113, 117, 139, 267, 273, 274, 278, 283, 386, 389, 393, 403, 411 and 416. Development tasks 3 and 101 are disjoint. Selection preceded formal scoring and used local service availability. Sixteen questions concern log-analysis workflows, limiting diversity. Official questions, resources and labels are not rewritten; some requested logs are absent, which is reported rather than repaired with invented data.

Tool scope follows official Early task/server metadata, retaining surviving named tools and newly added tools from those servers in later versions. Exposure means a scoped contract differs from Early, not that the changed tool is indispensable to solving the task.

Environment	Changed / unchanged tasks	Tool / parameter / description change exposure
Middle	9 / 9	3 / 8 / 9 tasks
Late	10 / 8	7 / 10 / 9 tasks

Categories overlap; these counts must not be summed as independent tasks.

Setting	Value
Actor and updater	Qwen3-4B-Instruct-2507 Q4_K_M
Independent local judge	Qwen2.5-7B-Instruct Q4_K_M
Backend / hardware	llama.cpp b11900; RTX 3060 Laptop 6 GB VRAM; 16 GB RAM
Temperature / seed / response limit	0 / 6203 / 2,048 tokens for each model role
Actor/updater context / decision rounds	16,384 tokens / maximum 12
Judge server context	24,576 tokens under the disclosed pre-scoring amendment
Per-round matrix	18 tasks x 3 policies x 3 versions = 162 episodes
Completed main experiments	3 rounds, 486 scored episodes; not 486 distinct tasks
Supplementary work	Original single 6-episode component variant and separate 9-episode development verification; excluded from main repeats

All models run locally; actor/updater and judge are loaded sequentially. API billing is zero, not zero hardware, electricity or development cost. The judge was calibrated on only two development trajectories; that is not general reliability validation.

Round	Scored / expected	Execution exceptions	Analysis status
R1: final-v1	162 / 162	4	Original round; included in R1+R3 comparison
R2: repeat-r2-20261029	162 / 162	5	First added round; separate due to input differences
R3: final-r3	162 / 162	5	Second added round; included in R1+R3 comparison

4. Metrics, comparability and integrity

TF is the original 1-10 task-fulfillment rubric score. PE is Planning Effectiveness in the course brief, represented upstream by planning_and_efficiency_mean; it evaluates redundant/unnecessary actions. ECS is the task-average of each task's mean TF minus its population standard deviation across the three versions. Original prompt formatting, aggregation and ECS functions are retained through a portability adapter for an unavailable import and fixed upstream MCP connection. A valid local score is not a verified task success.

R2 differs from R1 in 123 actual resource lists. All 54 same-task/same-version cross-policy groups in R2 also have resource-list differences, caused by Windows path-length-dependent filename visibility. Equal configuration files therefore do not establish equal inputs. R2 is retained and reported independently, not pooled or silently dropped. We do not claim that the input differences have been isolated as the cause of its score differences.

Before R3 scoring, the comparison rule was declared: include R1 and R3 only if actual inputs and runtime/scoring conditions pass verification. R3 used an equal-length output-directory name and passed 27 no-inference layout checks before execution. All 162 R3 resource lists match R1, and all 54 cross-policy groups in each of R1 and R3 agree. Recorded model, decoding and backend checks passed. Inclusion was based on inputs and declared conditions, not selecting rounds with favorable scores. This yields 324 observations, 18 distinct tasks, two observations per configuration. No three-round comparable summary is claimed.

Two historical differences remain disclosed. The first 39 original judgments used a 16,384-token client guard, later ones 24,576; the server was always 24,576. All 39 fit the earlier guard (largest reserved estimate 15,505), and all 162 original judge calls retain zero dropped messages. One original public ZIP result differs from its local frozen counterpart in JSON serialization while values and types agree. Both source hashes and a precise copy are retained. We do not claim every client implementation or serialized byte is identical.

Fixed seed and temperature-zero repetitions describe execution stability under the same settings, not independent new tasks or random sampling. Early has identical initial knowledge and no updating, yet its separate executions already differ. This weakens causal attribution of later policy gaps. All 14 execution exceptions remain included; completed actor episodes and completed scores are not retried to improve results. Score reconstruction uses only existing events and preserves failure originals.

5. Results

5.1 All three rounds, reported independently

Each stage mean below uses 18 episodes per policy. R2 is displayed despite its unfavorable selective results; its input mismatch prevents a controlled policy-effect interpretation.

R1: final-v1

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS
No update	6.83 / 6.56	7.28 / 7.33	7.44 / 7.89	6.280
Full rewrite	6.28 / 7.50	7.06 / 7.83	6.72 / 7.17	5.239
Selective patch	7.11 / 7.33	7.61 / 7.61	7.17 / 7.39	6.332

R2: repeat-r2-20261029 (input-mismatched, separate)

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS
No update	7.28 / 7.89	6.94 / 7.00	6.72 / 6.22	5.983
Full rewrite	6.72 / 6.56	6.61 / 7.28	6.89 / 7.11	5.611
Selective patch	7.17 / 7.39	6.28 / 6.33	5.78 / 7.17	4.597

R3: final-r3

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS
No update	6.06 / 6.94	7.50 / 7.22	6.78 / 6.61	4.929
Full rewrite	6.89 / 7.28	6.61 / 7.11	7.56 / 8.00	5.340
Selective patch	7.72 / 7.22	7.06 / 7.39	6.56 / 7.17	6.152

5.2 Declared R1+R3 descriptive comparison

Policy	Early TF / PE	Middle TF / PE	Late TF / PE	ECS mean +/- SD
No update	6.44 / 6.75	7.39 / 7.28	7.11 / 7.25	5.605 +/- 0.956
Full rewrite	6.58 / 7.39	6.83 / 7.47	7.14 / 7.58	5.289 +/- 0.071
Selective patch	7.42 / 7.28	7.33 / 7.50	6.86 / 7.28	6.242 +/- 0.127

SD is the sample standard deviation of the two round means (n=2), not a confidence interval, a standard deviation over 18 tasks or a significance test. ECS itself still uses the benchmark's population-SD formula within each task across versions. Stage means contain 36 repeated observations per policy/version, from 18 distinct tasks.

Selective patch has the highest ECS mean in these two comparable rounds, but its Late TF is 6.86, below full rewrite's 7.14 and no update's 7.11. Selective Early TF is already higher before any updating. The results do not establish a universal or causal method advantage. R2's selective ECS of 4.597 remains visible separately.

5.3 Recovery and retention

For changed-scope tasks, recovery is the paired TF difference from no update in the same version. For unchanged-scope tasks, retention is the paired TF difference from the policy's own Early score. Both signs are retained. A task can belong to different cohorts at Middle and Late. Each row below reports distinct tasks and repeated observations separately.

Policy	Version	Cohort	Tasks / observations	TF	Recovery / retention delta
No update	Middle	Changed	9 / 18	7.61	+0.00
No update	Middle	Unchanged	9 / 18	7.17	+0.78
No update	Late	Changed	10 / 20	7.40	+0.00
No update	Late	Unchanged	8 / 16	6.75	+0.81
Full rewrite	Middle	Changed	9 / 18	8.33	+0.72
Full rewrite	Middle	Unchanged	9 / 18	5.33	-0.44
Full rewrite	Late	Changed	10 / 20	7.55	+0.15
Full rewrite	Late	Unchanged	8 / 16	6.62	+1.38
Selective patch	Middle	Changed	9 / 18	7.94	+0.33
Selective patch	Middle	Unchanged	9 / 18	6.72	-0.44
Selective patch	Late	Changed	10 / 20	7.35	-0.05
Selective patch	Late	Unchanged	8 / 16	6.25	-0.81

For selective patch, changed-task recovery is +0.33 at Middle but -0.05 at Late; unchanged-task retention is -0.44 and -0.81. Full rewrite has +0.72/+0.15 recovery, but -0.44/+1.38 retention. These are mixed outcomes. Positive retention is not necessarily an update effect, since Early execution already varies and unchanged tools can coexist with actor/judge variation.

6. Update cost versus total recorded cost

The following totals use only the two comparable rounds. Seconds are cumulative request durations, not end-to-end elapsed time.

Policy	Phase	Calls	Tokens	Request seconds
No update	update	0	0	0.00
No update	agent	851	5,993,184	32,620.29
No update	judge	108	823,400	3,295.60
Full rewrite	update	72	208,898	660.07
Full rewrite	agent	804	5,338,700	15,384.36
Full rewrite	judge	108	795,326	4,712.60
Selective patch	update	38	72,898	235.25
Selective patch	agent	810	5,832,127	9,206.16
Selective patch	judge	108	794,178	5,698.87

Policy	Update tokens	All three phases: tokens
No update	0	6,816,584
Full rewrite	208,898	6,342,924
Selective patch	72,898	6,699,203

Selective patch uses 72,898 update tokens versus 208,898, saving 65.1%. This is the main supported efficiency result. Total recorded tokens are 6,699,203 versus 6,342,924, increasing 5.6%, because actor and judge work dominate. R3 alone has approximately 3.5% higher total selective tokens; R2 separately has approximately 8.4% more. Update savings must not be described as total-token or total-compute savings.

All local API charges are zero. Hardware, electricity and development cost are not monetized. Calls interrupted before returning usage have unknown cost; zero missing cost is not assumed. Original scoring experienced allocation interruptions after 39 and 57 saved judgments. Their logs and same-configuration resumptions retain completed score hashes and the unknown unreturned-request costs.

7. Failure analysis and judge disagreement

7.1 Under-update: required parameters, task 113

Early planning requires originalRequest and tasks; Middle adds splitDetails; Late also requires priority and tags. In R3 no-update Late, the first planning call omits the new fields. Six further planning calls supply splitDetails but still omit priority and tags. All seven fail required-field validation; the root planning request is not registered. An empty project task store is not evidence of a successful plan. This is direct stale-parameter evidence and the no-update limiting case of under-update, not proof that selective patch omitted those fields.

R3 full rewrite and selective patch Late supply all five required fields and save plans. The selective actor also supplies estimatedDeadline. Saved tasks remain unfinished/unapproved; this establishes plan registration, not implemented dashboard code, tests or genuine human approval. R3 no-update Late nevertheless receives TF/PE 10/10, while selective Middle/Late receive TF 4. Original scores are preserved, so successful registration cannot be equated with measured TF recovery or dashboard completion.

7.2 Over-update candidate: unchanged guidance, task 28

The task compares two logs and requires saved to-do items. R3 full rewrite Middle has no scoped contract changes but regenerates all three notes. It then invents source_id_1 for task addition and tries to initialize two .txt logs as task sources, producing one missing-source and two unsupported-format errors. No target task file is created. Selective Middle preserves the notes and writes tasks through a valid initialization/addition sequence.

R3 full rewrite Late again rewrites all notes despite only the reader contract changing, and fails to save the target tasks. Selective Late recovers from one parameter error and saves a task file after four calls. These observations associate unnecessary rewriting with regression, but do not isolate a sentence-level cause: the actor also disobeys guidance. The original round contains a related over-update candidate and counterexamples. Some successfully written tasks overstate an unresolved database fault absent from today's log; file existence alone does not prove grounded content.

7.3 Other errors and misleading scores

R3 task 117 selective Early stops with an exception, an empty answer and no task file, yet receives 10/10. It occurs before any updating, so cannot be a selective-update failure. Full rewrite Middle has three split_tasks returns explicitly describing invalid JSON while isError is false, no saved Shrimp tasks, and also receives 10/10. Full rewrite Late fails at its third call and receives TF/PE 8/10. Flagged-error counts therefore miss semantic errors, and high scores do not establish actual completion.

R3 selective Middle task 117 does save six tasks, but then uses a task ID as another tool system's source_id; both history searches fail. Identifier types can match while identifier provenance and workflow ordering are wrong. A successful empty query of work.md in no-update Late also does not mean reusable historical experience was found. No task storage, no successful search, and a normal empty result are distinct outcomes.

R3 execution exception	Task	TF / PE
Full rewrite Late	117	8 / 10
No update Middle	389	7 / 4
Selective patch Late	416	7 / 7
Selective patch Early	117	10 / 10
Selective patch Early	389	8 / 4

All five have empty final answers and remain included. Generic infrastructure_error/ExceptionGroup labels do not establish hardware root causes. We separate stale knowledge, unrelated-note changes, actor planning/ID errors, serialization/tool faults, absent resources and judge disagreement. We do not replace automated scores with favorable manual labels or discard failures. Concrete calls, returned text and saved artifacts support each interpretation.

7.4 Supplementary dependency-expansion comparison

The original single component variant disables dependency expansion on tasks 113 and 416 across three versions, adding six episodes outside main repeats. The original expanded method is lower by 6 TF points on task 113 Middle and task 416 Late, tied on two other changed-stage pairs; an Early difference also occurs before any expansion. This small, single comparison does not establish expansion's benefit or a controlled causal disadvantage. It was not repeated, and is not included among the 486 main scores.

8. Discussion and limits

Selective updating can reduce note-generation work and avoid rewriting unaffected knowledge. It is most plausible when public changes are localized and usefully described by explicit dependencies. It can fail when usage rules are implicit, generated notes are incomplete, the actor violates correct guidance, resources are unavailable or the judge rewards unsupported plans. No-update can remain competitive when updated tools are not needed, or measured scores fail to detect actual errors.

Our quality evidence is limited by 18 tasks dominated by log workflows, small quantized models, two comparable fixed-setting repeats and a fallible local judge. Early imbalance, output paths, generated IDs and numeric implementation behavior prevent treating score gaps as isolated update effects. Timing is machine/order/cache dependent. We claim neither significance nor state-of-the-art performance. More diverse tasks, independently designed repeats, grounded outcome checks and genuine human assessment would be needed for stronger conclusions.

The engineering lesson is to check valid calls, correctly sourced IDs, prerequisites and persisted outputs after a tool update. A fluent answer, a successful MCP flag or a high judge score alone is insufficient. Lower update cost and recovery/retention should be assessed separately.

9. Reproduction, evidence and contribution status

Installation, frozen downloads, model startup and original-pipeline commands are documented in the reproduction guide. The original pipeline produces one 162-episode grid and the six-episode component variant; added-round scripts implement prechecks, sequential execution, guarded continuation and evidence publication. Existing completed directories must not be rerun or overwritten. Independent audits recompute completeness and arithmetic; protocol checks do not establish answer correctness. Separate development verification is not another main repeat.

Three public Releases retain the original evidence packages. The R2 and R3 ZIPs were anonymously downloaded and checked for all entry sizes, SHA hashes and CRC. R3 has 909 manifest entries, including 162 results, 162 actor copies, 162 event logs and five failure originals. Later pair analyses and scored-case reviews are branch supplements, not falsely represented as original ZIP entries. Ten intermediate progress receipts and two old development sources have only been relocated into documented archives; historical receipt paths are not rewritten.

Code development, report preparation and evidence checks used AI assistance. The project README lists the six members and their responsibility areas. Scientific conclusions are supported by the saved experiment sources rather than inferred from file ownership or upload activity.

10. Conclusion

We built and evaluated a selective tool-knowledge adaptation system using unchanged local models and actual evolving MCP environments. Three rounds are fully scored and preserved, with one input-mismatched round kept separate. In the declared two-round comparison, selective updating reduces update tokens by 65.1%, but increases recorded total tokens by 5.6% and does not consistently improve recovery or retention. Its higher observed ECS mean is descriptive. Grounded failure evidence and transparent comparability limits are as central to this result as the aggregate scores.

Evidence and sources

1. **Course brief.** PE6203 Group Project 4: Tool Harness Adaptation under Environment Change (provided document).

2. **Benchmark.** Pinned MCPEvol-Bench.

3. **Comparable metrics.** R1+R3 metrics, costs, cohorts and limits

4. **Independent rounds.** R2 metrics; R3 metrics

5. **Paired provenance.** 162 matched pairs and source hashes

6. **Input fairness.** Final cross-policy resource checks

7. **Context and serialization.** Original context audit; Original public evidence

8. **Scored cases.** R2 case review; R3 case review; R3 five exceptions

9. **Evidence downloads.** Original release; R2 release; R3 release.

10. **Official servers.** Official archive dataset.

11. **Model sources.** Actor/updater; judge; llama.cpp b11900.